

LAPORAN TUGAS FINAL
STRUKTUR DATA TEORI

SILOG - SISTEM INFORMASI LOGISTIK



Disusun oleh:

Syayidati Hapsari

NIM: 5250411216

FAKULTAS SAINS DAN TEKNOLOGI
PROGRAM STUDI INFORMATIKA

2025 / 2026

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam era digitalisasi saat ini, sistem logistik memerlukan pengelolaan data yang efisien dan terstruktur. Permasalahan umum yang dihadapi mencakup pencarian rute pengiriman terpendek, pengelolaan data resi, serta penjadwalan kurir. Proyek SILOG (Sistem Informasi Logistik) hadir sebagai solusi berbasis konsep struktur data untuk menjawab tantangan tersebut.

Program ini dikembangkan menggunakan bahasa pemrograman Python dengan menerapkan tiga struktur data utama, yaitu Weighted Undirected Graph, Binary Search Tree (BST), serta Array dan Dictionary. Ketiga struktur data ini dipilih sesuai dengan karakteristik kebutuhan masing-masing fitur pada sistem logistik.

1.2 Tujuan

Tujuan dari proyek final ini adalah sebagai berikut:

1. Menerapkan struktur data Weighted Graph untuk merepresentasikan jaringan kota hub dan rute antar kota.
2. Menerapkan Binary Search Tree (BST) untuk menyimpan dan mencari data resi pengiriman.
3. Menerapkan algoritma Selection Sort (Descending) untuk mengurutkan data transaksi.
4. Menerapkan algoritma Dijkstra untuk menghitung jarak terpendek antar kota.
5. Membangun sistem informasi logistik yang terintegrasi dengan antarmuka berbasis teks (CLI).

1.3 Batasan Masalah

Sesuai ketentuan NIM digit terakhir genap (6), program ini memiliki batasan sebagai berikut:

1. Nomor resi wajib diawali dengan angka '2' dan terdiri dari tepat 4 digit.
2. Algoritma pengurutan yang digunakan adalah Selection Sort secara descending.
3. Tiga kota hub default yang wajib terdaftar adalah SURABAYA, YOGYAKARTA, dan MAGELANG (mengandung 3 huruf pertama nama depan: SYA).
4. Tarif pengiriman dihitung dengan formula: $(\text{Jarak} \times \text{Rp}2.000) + (\text{Berat} \times \text{Rp}5.000)$.

BAB II

LANDASAN TEORI

2.1 Weighted Undirected Graph

Graf berbobot tidak berarah (Weighted Undirected Graph) adalah struktur data yang merepresentasikan hubungan antar simpul (node) di mana setiap sisi (edge) memiliki nilai bobot dan bersifat dua arah. Dalam program ini, kota-kota hub direpresentasikan sebagai simpul, sedangkan rute antar kota direpresentasikan sebagai sisi dengan bobot berupa jarak dalam kilometer.

2.2 Algoritma Dijkstra

Algoritma Dijkstra adalah algoritma pencarian jalur terpendek pada graf berbobot. Algoritma ini bekerja dengan cara menjelajahi simpul secara greedy, selalu mengambil simpul dengan jarak terkecil yang belum diproses. Dijkstra digunakan dalam SILOG untuk menghitung jarak pengiriman terpendek antara kota asal dan kota tujuan, yang kemudian menjadi dasar perhitungan biaya.

2.3 Binary Search Tree (BST)

Binary Search Tree adalah struktur data pohon di mana setiap simpul memiliki paling banyak dua anak. Nilai pada subpohon kiri selalu lebih kecil dari simpul induk, dan nilai subpohon kanan selalu lebih besar. BST memungkinkan operasi insert dan search dengan kompleksitas rata-rata $O(\log n)$. Dalam SILOG, BST digunakan untuk menyimpan dan mencari data resi berdasarkan nomor resi.

2.4 Selection Sort

Selection Sort adalah algoritma pengurutan yang bekerja dengan cara mencari elemen maksimum (untuk descending) dari sisa larik yang belum terurut, kemudian menukarnya ke posisi yang tepat. Kompleksitas waktu algoritma ini adalah $O(n^2)$. Dalam program ini, Selection Sort diterapkan untuk mengurutkan daftar resi berdasarkan biaya pengiriman dari terbesar ke terkecil.

2.5 Dataclass Python

Dataclass adalah fitur Python (modul dataclasses) yang memungkinkan pembuatan kelas data secara ringkas. Atribut kelas dideklarasikan langsung di level kelas dengan anotasi tipe, dan Python secara otomatis membuat metode `__init__`, `__repr__`, dan lainnya. Program ini menggunakan dataclass untuk mendefinisikan struktur data Receipt dan Courier.

BAB III

ANALISIS PROGRAM

3.1 Struktur Umum Program

Program SILOG terdiri dari beberapa lapisan, yaitu lapisan konfigurasi dan konstanta, lapisan struktur data (dataclass), lapisan implementasi struktur data inti (BST dan Graph), lapisan fungsi utilitas, lapisan fungsi sorting, lapisan menu sistem, dan fungsi utama main(). Seluruh data program disimpan dalam variabel global yang dapat diakses oleh semua fungsi menu.

```
# SILOG - Sistem Informasi Logistik
# Tugas Final Struktur Data
# Nama : SYAYIDATI HAPSARI
# NIM : 5250411216 (digit terakhir: 6 -> GENAP)
#
# Fitur:
# 1. Weighted Undirected Graph (Adjacency List via Dictionary)
# 2. Binary Search Tree for Resi
# 3. Array/List + Dictionary untuk Kurir & Manifest
#
# Catatan (NIM GENAP):
# - No. resi wajib diawali '2'
# - Algoritma sorting: Selection Sort (descending)
# - kota hub default: SURABAYA, YOGYAKARTA
# (saalah satunya mengandung 3 huruf pertama nama depan: SYA)

from __future__ import annotations

import heapq
import re
from dataclasses import dataclass
from typing import Optional, List, Dict, Tuple

# =====
# KONFIGURASI
# =====
LAST_NIM_DIGIT = 6
RATE_PER_KM = 2000
RATE_PER_KG = 5000
DEFAULT_CITIES = ["SURABAYA", "YOGYAKARTA"]
```

```

# =====
# DATA CLASS
# =====
@dataclass
class Receipt:
    no_resi: str
    sender: str
    origin: str
    destination: str
    weight: float
    distance_km: float
    total_cost: float

@dataclass
class Courier:
    courier_id: str
    name: str
    vehicle_type: str

# =====
# TABEL HELPER
# =====
def print_table(headers: List[str], rows: List[List[str]], col_widths:
List[int] = None, indent: int = 0):
    """Cetak tabel dengan border ASCII. indent = jumlah spasi menjorok
    ke kanan."""
    if not col_widths:
        col_widths = [max(len(str(headers[i])), max((len(str(r[i])) for
r in rows), default=0))
                        for i in range(len(headers))]

    pad = " " * indent

    def row_line(cells):
        return pad + "|" + " | ".join(str(c).ljust(col_widths[i]) for
i, c in enumerate(cells)) + " |"

    sep = pad + "+" + "+".join("-" * (w + 2) for w in col_widths) + "+"

    print(sep)
    print(row_line(headers))
    print(sep)

```

```

    for row in rows:
        print(row_line(row))
    print(sep)

# =====
# BST UNTUK RESI
# =====
class BSTNode:
    def __init__(self, data: Receipt):
        self.data = data
        self.left: Optional["BSTNode"] = None
        self.right: Optional["BSTNode"] = None

class ReceiptBST:
    def __init__(self):
        self.root: Optional[BSTNode] = None

    def insert(self, data: Receipt) -> bool:
        if self.root is None:
            self.root = BSTNode(data)
            return True
        curr = self.root
        while True:
            if data.no_resi == curr.data.no_resi:
                return False
            elif data.no_resi < curr.data.no_resi:
                if curr.left is None:
                    curr.left = BSTNode(data)
                    return True
                curr = curr.left
            else:
                if curr.right is None:
                    curr.right = BSTNode(data)
                    return True
                curr = curr.right

    def search(self, no_resi: str) -> Optional[Receipt]:
        curr = self.root
        while curr is not None:
            if no_resi == curr.data.no_resi:
                return curr.data
            elif no_resi < curr.data.no_resi:
                curr = curr.left

```

```

        else:
            curr = curr.right
        return None

def inorder(self) -> List[Receipt]:
    result: List[Receipt] = []
    def _traverse(node: Optional[BSTNode]):
        if not node:
            return
        _traverse(node.left)
        result.append(node.data)
        _traverse(node.right)
    _traverse(self.root)
    return result

def to_list(self) -> List[Receipt]:
    return self.inorder()

# =====
# GRAPH BERBOBOT TIDAK BERARAH
# =====
class WeightedGraph:
    def __init__(self):
        self.adj: Dict[str, Dict[str, float]] = {}

    @staticmethod
    def normalize_city(city: str) -> str:
        return city.strip().upper()

    def add_city(self, city: str) -> bool:
        city = self.normalize_city(city)
        if not city:
            return False
        if city in self.adj:
            return False
        self.adj[city] = {}
        return True

    def add_route(self, city1: str, city2: str, distance: float) ->
    Tuple[bool, str]:
        city1 = self.normalize_city(city1)
        city2 = self.normalize_city(city2)
        if city1 == city2:
            return False, "Kota asal dan tujuan tidak boleh sama."

```

```

        if city1 not in self.adj or city2 not in self.adj:
            return False, "Pastikan kedua kota sudah terdaftar."
        if distance <= 0:
            return False, "Jarak harus lebih besar dari 0."
        self.adj[city1][city2] = distance
        self.adj[city2][city1] = distance
        return True, "Rute berhasil ditambahkan."

def has_city(self, city: str) -> bool:
    return self.normalize_city(city) in self.adj

def is_connected(self, start: str, end: str) -> bool:
    return self.shortest_distance(start, end) is not None

def shortest_distance(self, start: str, end: str) ->
Optional[float]:
    """Dijkstra: mengembalikan jarak terpendek; None jika tak
    terhubung."""
    start = self.normalize_city(start)
    end = self.normalize_city(end)
    if start not in self.adj or end not in self.adj:
        return None
    if start == end:
        return 0.0
    dist = {node: float("inf") for node in self.adj}
    dist[start] = 0.0
    pq = [(0.0, start)]
    visited = set()
    while pq:
        curr_dist, node = heapq.heappop(pq)
        if node in visited:
            continue
        visited.add(node)
        for nxt, weight in self.adj[node].items():
            new_dist = curr_dist + weight
            if new_dist < dist[nxt]:
                dist[nxt] = new_dist
                heapq.heappush(pq, (new_dist, nxt))
    return dist[end] if dist[end] != float("inf") else None

def show_cities(self) -> None:
    if not self.adj:
        print("Belum ada kota yang terdaftar.")
        return
    print("\nDaftar Kota Hub:")

```



```

        rows = [[str(i), city] for i, city in enumerate(self.adj.keys(),
1)]

        print_table(["No", "Nama Kota"], rows, [4, 20])

    def show_routes(self) -> None:
        if not self.adj:
            print("Belum ada data graph.")
            return
        print("\nDaftar Rute Antar-Kota:")
        rows = []
        for i, city in enumerate(self.adj.keys(), 1):
            if not self.adj[city]:
                rows.append([str(i), city, "-", "Belum ada rute"])
            else:
                for j, (neighbor, dist) in
enumerate(self.adj[city].items()):
                    if j == 0:
                        rows.append([str(i), city, neighbor, f"{dist:g}
KM"])
                    else:
                        rows.append(["", "", neighbor, f"{dist:g} KM"])
        print_table(["No", "Kota Asal", "Kota Tujuan", "Jarak"], rows,
[4, 15, 15, 12])

# =====
# UTILITAS
# =====
def clear_screen():
    print("\n" + "=" * 70)

def pause():
    input("\nTekan ENTER untuk kembali...")

def input_nonempty(prompt: str) -> str:
    while True:
        value = input(prompt).strip()
        if value:
            return value
        print("Input tidak boleh kosong.")

def input_float(prompt: str, minimum: Optional[float] = None) -> float:
    while True:

```

```

    try:
        value = float(input(prompt).strip())
        if minimum is not None and value < minimum:
            print(f"Nilai harus >= {minimum}.")
            continue
        return value
    except ValueError:
        print("Masukkan angka yang valid.")

def input_int(prompt: str, minimum: Optional[int] = None, maximum:
Optional[int] = None) -> int:
    while True:
        try:
            value = int(input(prompt).strip())
            if minimum is not None and value < minimum:
                print(f"Nilai harus >= {minimum}.")
                continue
            if maximum is not None and value > maximum:
                print(f"Nilai harus <= {maximum}.")
                continue
            return value
        except ValueError:
            print("Masukkan bilangan bulat yang valid.")

def ask_yes_no(prompt: str) -> bool:
    while True:
        choice = input(prompt).strip().upper()
        if choice in ("Y", "N"):
            return choice == "Y"
        print("Masukkan Y atau N.")

def format_money(amount: float) -> str:
    return f"Rp {amount:,.0f}".replace(",", ".")

def calculate_badge_and_bonus(package_count: int) -> Tuple[str, int]:
    if package_count == 0:
        return "Kurir Santai", 0
    if 1 <= package_count <= 3:
        return "Kurir Reguler", 25000
    if 4 <= package_count <= 6:
        return "Kurir Produktif", 60000
    return "Kurir Elite", 120000

```

```

def valid_resi_format(no_resi: str) -> bool:
    if not re.fullmatch(r"\d{4}", no_resi):
        return False
    return no_resi.startswith("2")

# =====
# SORTING - SELECTION SORT DESCENDING
# =====
def selection_sort_desc(items: List[Receipt]) -> List[Receipt]:
    arr = items[:]
    n = len(arr)
    for i in range(n):
        max_idx = i
        for j in range(i + 1, n):
            if arr[j].total_cost > arr[max_idx].total_cost:
                max_idx = j
        arr[i], arr[max_idx] = arr[max_idx], arr[i]
    return arr

# =====
# MENU SISTEM
# =====
graph = WeightedGraph()
bst = ReceiptBST()
couriers: List[Courier] = []
manifest: Dict[str, List[str]] = {}
assigned_resi: set[str] = set()

def seed_default_cities():
    print("Mendaftarkan kota-kota hub default ke dalam graph...")
    for city in DEFAULT_CITIES:
        if graph.add_city(city):
            print(f" -> Kota '{city}' berhasil ditambahkan.")
        else:
            print(f" -> Kota '{city}' sudah ada, dilewati.")
    print(f"Total {len(DEFAULT_CITIES)} kota hub default telah terdaftar.\n")

def menu_1():
    while True:

```

```

clear_screen()
print("MENU 1) KELOLA JARINGAN HUB DAN RUTE LOGISTIK")
print("1.1) Input Hub Kota Baru")
print("1.2) Input Rute Antar-Kota")
print("1.3) Tampil Data Hub & Rute")
print("0) Kembali ke MENU UTAMA")

choice = input("Pilih menu: ").strip()
if choice == "1.1":
    input_city_menu()
elif choice == "1.2":
    input_route_menu()
elif choice == "1.3":
    graph.show_cities()
    graph.show_routes()
    pause()
elif choice == "0":
    return
else:
    print("Pilihan tidak valid.")
    pause()

def input_city_menu():
    while True:
        clear_screen()
        city = input_nonempty("Masukkan nama kota hub baru: ")
        if graph.add_city(city):
            print("Hub kota berhasil ditambahkan.")
        else:
            print("Kota sudah ada atau input tidak valid.")
        if not ask_yes_no("Apakah ingin menambahkan kota lagi (Y/N)? "):
            break

def input_route_menu():
    while True:
        clear_screen()
        if len(graph.adj) < 2:
            print("Minimal harus ada 2 kota untuk menambahkan rute.")
            pause()
            return
        graph.show_cities()
        c1 = input_nonempty("Masukkan kota asal: ")
        c2 = input_nonempty("Masukkan kota tujuan: ")

```

```

        dist = input_float("Masukkan jarak rute (KM): ",
minimum=0.000001)
        ok, msg = graph.add_route(c1, c2, dist)
        print(msg)
        if not ask_yes_no("Apakah ingin menambahkan rute lagi (Y/N)? "):
            break

def input_receipt_menu():
    while True:
        clear_screen()
        print("INPUT RESI PENGIRIMAN BARU")
        if len(graph.adj) < 2:
            print("Belum cukup kota hub untuk validasi rute
pengiriman.")
            pause()
            return

        no_resi = input_nonempty("Masukkan No. Resi (4 digit, diawali
'2'): ")
        if not valid_resi_format(no_resi):
            print("No. resi harus 4 digit dan diawali dengan '2'.")
            if not ask_yes_no("Coba input resi lagi (Y/N)? "):
                return
            continue

        if bst.search(no_resi) is not None:
            print("No. resi sudah terdaftar.")
            if not ask_yes_no("Coba input resi lagi (Y/N)? "):
                return
            continue

        sender = input_nonempty("Nama Pengirim: ")
        origin = input_nonempty("Kota Asal: ").upper()
        destination = input_nonempty("Kota Tujuan: ").upper()
        weight = input_float("Berat Paket (Kg): ", minimum=0.000001)

        if not graph.has_city(origin) or not
graph.has_city(destination):
            print("Pastikan kota asal dan kota tujuan sudah terdaftar di
graph.")
            if not ask_yes_no("Coba input resi lagi (Y/N)? "):
                return
            continue

```

```

distance = graph.shortest_distance(origin, destination)
if distance is None:
    print("Rute pengiriman belum tersedia dalam jaringan!")
    if not ask_yes_no("Coba input resi lagi (Y/N)? "):
        return
    continue

total_cost = (distance * RATE_PER_KM) + (weight * RATE_PER_KG)
data = Receipt(
    no_resi=no_resi,
    sender=sender,
    origin=origin,
    destination=destination,
    weight=weight,
    distance_km=distance,
    total_cost=total_cost,
)

if bst.insert(data):
    print("\nInput Resi Berhasil!")
    print_table(
        ["No Resi", "Pengirim", "Asal", "Tujuan", "Berat (Kg)",
        "Jarak (KM)", "Total Biaya"],
        [[data.no_resi, data.sender, data.origin,
        data.destination,
        f"{data.weight:g}", f"{data.distance_km:g}",
        format_money(data.total_cost)]],
        [8, 15, 12, 12, 10, 10, 16]
    )
else:
    print("Gagal menyimpan resi karena no. resi sudah ada.")

    if not ask_yes_no("Apakah ingin melakukan input resi lagi (Y/N)?
    "):
        break

def show_all_receipts_menu():
    clear_screen()
    data = bst.inorder()
    print("DATA RESI TERDAFTAR")
    if not data:
        print("Belum ada data resi.")
        pause()
    return

```

```

        rows = [
            [r.no_resi, r.sender, r.origin, r.destination,
             f"{r.weight:g}", f"{r.distance_km:g}",
             format_money(r.total_cost)]
            for r in data
        ]
        print_table(
            ["No Resi", "Pengirim", "Asal", "Tujuan", "Berat (Kg)", "Jarak
            (KM)", "Total Biaya"],
            rows,
            [8, 15, 12, 12, 10, 10, 16]
        )
        pause()

def sort_receipts_menu():
    clear_screen()
    data = bst.to_list()
    print("URUTAN TRANSAKSI RESI BERDASARKAN BIAYA TERBESAR")
    if not data:
        print("Belum ada data resi.")
        pause()
        return

    sorted_data = selection_sort_desc(data)
    rows = [
        [str(i), r.no_resi, format_money(r.total_cost), r.sender,
         r.origin, r.destination, f"{r.weight:g}"]
         for i, r in enumerate(sorted_data, 1)
        ]
    print_table(
        ["Rank", "No Resi", "Total Biaya", "Pengirim", "Asal", "Tujuan",
        "Berat (Kg)"],
        rows,
        [4, 8, 16, 15, 12, 12, 10]
    )
    pause()

def menu_2():
    while True:
        clear_screen()
        print("MENU 2) KELOLA ADMINISTRASI RESI PENGIRIMAN")
        print("2.1) Input Resi Pengiriman Baru")

```

```

print("2.2) Lihat Seluruh Data Resi Terdaftar")
print("2.3) Urutkan Transaksi Resi Berdasarkan Biaya Terbesar")
print("0) Kembali ke MENU UTAMA")

choice = input("Pilih menu: ").strip()
if choice == "2.1":
    input_receipt_menu()
elif choice == "2.2":
    show_all_receipts_menu()
elif choice == "2.3":
    sort_receipts_menu()
elif choice == "0":
    return
else:
    print("Pilihan tidak valid.")
    pause()

def input_courier_menu():
    while True:
        clear_screen()
        print("INPUT DATA KURIR")
        courier_id = input_nonempty("ID Kurir: ")
        if any(c.courier_id == courier_id for c in couriers):
            print("ID kurir sudah terdaftar.")
            if not ask_yes_no("Apakah ingin menambahkan data Kurir lagi
(Y/N)? "):
                break
            continue

        name = input_nonempty("Nama Kurir: ")
        vehicle = input_nonempty("Jenis Kendaraan (Motor/Mobil/Truck):
").capitalize()
        if vehicle not in ("Motor", "Mobil", "Truck"):
            print("Jenis kendaraan harus salah satu dari: Motor, Mobil,
Truck.")
            if not ask_yes_no("Apakah ingin menambahkan data Kurir lagi
(Y/N)? "):
                break
            continue

        couriers.append(Courier(courier_id, name, vehicle))
        manifest.setdefault(courier_id, [])
        print("Data Kurir Berhasil Diinputkan")
        if not ask_yes_no("Apakah ingin menambahkan data Kurir lagi

```



```

(Y/N)? "):
    break

def show_available_receipts():
    data = bst.inorder()
    if not data:
        print("Belum ada data resi.")
        return []

    print("\nDaftar Resi:")
    rows = [
        [r.no_resi, r.origin, r.destination,
         format_money(r.total_cost),
         "Sudah dialokasikan" if r.no_resi in assigned_resi else
         "Tersedia"]
        for r in data
    ]
    print_table(
        ["No Resi", "Asal", "Tujuan", "Total Biaya", "Status"],
        rows,
        [8, 12, 12, 16, 18]
    )
    return data

def plotting_manifest_menu():
    clear_screen()
    print("PLOTING PENUGASAN MANIFEST")
    if not couriers:
        print("Belum ada data kurir.")
        pause()
        return
    if not bst.inorder():
        print("Belum ada data resi.")
        pause()
        return

    print("Daftar Kurir:")
    rows_kurir = [[c.courier_id, c.name, c.vehicle_type] for c in
couriers]
    print_table(["ID Kurir", "Nama", "Kendaraan"], rows_kurir, [10, 20,
12])

    courier_id = input_nonempty("\nMasukkan ID Kurir yang akan

```

```

ditugaskan: ")
    target = next((c for c in couriers if c.courier_id == courier_id),
None)
    if target is None:
        print("ID kurir tidak ditemukan.")
        pause()
        return

    print(f"\nKurir terpilih: {target.name} ({target.vehicle_type}")
    print("Silakan pilih resi yang akan dimasukkan ke manifest.")
    while True:
        available = show_available_receipts()
        if not available:
            pause()
            return

        no_resi = input_nonempty("\nMasukkan No. Resi yang akan
ditugaskan: ")
        receipt = bst.search(no_resi)
        if receipt is None:
            print("No. resi tidak ditemukan.")
        elif no_resi in assigned_resi:
            print("Resi sudah ditugaskan ke kurir lain.")
        else:
            manifest.setdefault(courier_id, []).append(no_resi)
            assigned_resi.add(no_resi)
            print("Resi berhasil ditambahkan ke manifest kurir.")

        if not ask_yes_no("Apakah ingin memasukkan No. Resi lagi (Y/N)?
"):
            break

    print("Proses plotting manifest selesai.")
    pause()

def tampil_manifest_menu():
    clear_screen()
    print("TAMPIL MANIFEST & ATURAN BONUS INSENTIF")
    if not couriers:
        print("Belum ada data kurir.")
        pause()
        return

    receipt_cache = {r.no_resi: r for r in bst.inorder()}

```

```

for i, c in enumerate(couriers, 1):
    resi_list = manifest.get(c.courier_id, [])
    badge, bonus = calculate_badge_and_bonus(len(resi_list))

    print(f"\n{'=' * 75}")
    print_table(
        ["No", "ID Kurir", "Nama Kurir", "Kendaraan", "Jml Resi",
"Lencana", "Bonus Insentif"],
        [[str(i), c.courier_id, c.name, c.vehicle_type,
            str(len(resi_list)), badge, format_money(bonus)]],
        [4, 10, 20, 10, 9, 18, 14]
    )

    if not resi_list:
        print("      Manifest kosong.")
    else:
        print("      Rincian Resi:")
        detail_rows = []
        for idx, no_resi in enumerate(resi_list, 1):
            r = receipt_cache.get(no_resi)
            if r is None:
                detail_rows.append([str(idx), no_resi, "-", "-", "-",
", "-", "-"])
            else:
                detail_rows.append([
                    str(idx), r.no_resi, r.sender,
                    r.origin, r.destination,
                    f"{r.weight:g} Kg", format_money(r.total_cost)
                ])
        print_table(
            ["No", "No Resi", "Pengirim", "Asal", "Tujuan", "Berat",
"Total Biaya"],
            detail_rows,
            [4, 8, 18, 12, 12, 8, 16],
            indent=6
        )

    print()
    pause()

def menu_3():
    while True:
        clear_screen()

```

```

        print("MENU 3) KELOLA KURIR DAN MANIFEST PENGANTARAN")
        print("3.1) Input Data Kurir")
        print("3.2) Plotting Penugasan Manifest")
        print("3.3) Tampil Manifest & Aturan Bonus Insentif")
        print("0) Kembali ke MENU UTAMA")

        choice = input("Pilih menu: ").strip()
        if choice == "3.1":
            input_courier_menu()
        elif choice == "3.2":
            plotting_manifest_menu()
        elif choice == "3.3":
            tampil_manifest_menu()
        elif choice == "0":
            return
        else:
            print("Pilihan tidak valid.")
            pause()

def main():
    seed_default_cities()

    while True:
        clear_screen()
        print("SILOG - SISTEM INFORMASI LOGISTIK")
        print("-" * 70)
        print("1) Kelola Jaringan Hub dan Rute Logistik")
        print("2) Kelola Administrasi Resi Pengiriman")
        print("3) Kelola Kurir dan Manifest Pengantaran")
        print("0) Exit Program")

        choice = input("Pilih menu utama: ").strip()
        if choice == "1":
            menu_1()
        elif choice == "2":
            menu_2()
        elif choice == "3":
            menu_3()
        elif choice == "0":
            print("Program dihentikan.")
            break
        else:
            print("Pilihan tidak valid.")
            pause()

```

```
if __name__ == "__main__":  
    main()
```

3.2 Data Class

Program mendefinisikan dua dataclass sebagai model data utama:

a. Receipt

Merupakan representasi data resi pengiriman. Setiap objek Receipt menyimpan atribut: no_resi (nomor resi unik 4 digit berawalan '2'), sender (nama pengirim), origin (kota asal), destination (kota tujuan), weight (berat paket dalam kg), distance_km (jarak pengiriman dalam km yang dihitung otomatis oleh Dijkstra), dan total_cost (total biaya yang dihitung otomatis).

b. Courier

Merupakan representasi data kurir. Setiap objek Courier menyimpan atribut: courier_id (ID unik kurir), name (nama kurir), dan vehicle_type (jenis kendaraan: Motor, Mobil, atau Truck).

3.3 Class BSTNode dan ReceiptBST

Dua kelas ini bekerja bersama untuk mengimplementasikan Binary Search Tree bagi penyimpanan data resi.

a. BSTNode

Merupakan unit terkecil dari BST. Setiap node menyimpan satu objek Receipt sebagai data, serta referensi ke anak kiri (left) dan anak kanan (right). Perbandingan posisi node dilakukan berdasarkan nilai string no_resi secara leksikografis.

b. ReceiptBST

Merupakan kelas pengelola BST yang menyimpan referensi ke root. Kelas ini memiliki tiga metode utama:

1. insert(data): Menyisipkan objek Receipt baru ke dalam BST. Metode ini menelusuri pohon secara iteratif dan menempatkan node sesuai urutan no_resi. Jika no_resi sudah ada, operasi dibatalkan dan mengembalikan False.

2. `search(no_resi)`: Mencari resi berdasarkan `no_resi` secara iteratif. Mengembalikan objek `Receipt` jika ditemukan, atau `None` jika tidak ada.
3. `inorder()`: Melakukan traversal in-order (kiri → akar → kanan) secara rekursif sehingga menghasilkan daftar resi yang terurut secara alfabetis berdasarkan `no_resi`.

3.4 Class `WeightedGraph`

Mengimplementasikan `Weighted Undirected Graph` menggunakan adjacency list berbasis dictionary. Kunci utama dictionary adalah nama kota (string), dan nilainya adalah dictionary berisi tetangga beserta jaraknya. Kelas ini memiliki beberapa metode:

1. `normalize_city(city)`: Metode statis yang menormalisasi nama kota menjadi huruf kapital dan menghapus spasi ekstra, sehingga input tidak sensitif terhadap kapitalisasi.
2. `add_city(city)`: Mendaftarkan kota baru ke dalam graph. Kota yang sudah terdaftar atau input kosong akan ditolak.
3. `add_route(city1, city2, distance)`: Menambahkan rute dua arah (undirected) antara dua kota dengan bobot jarak tertentu. Validasi dilakukan untuk memastikan kedua kota sudah terdaftar, bukan kota yang sama, dan jarak bernilai positif.
4. `has_city(city)`: Mengecek apakah suatu kota sudah terdaftar dalam graph.
5. `shortest_distance(start, end)`: Menghitung jarak terpendek menggunakan algoritma Dijkstra dengan priority queue (heapq). Mengembalikan nilai float jika ada jalur, atau `None` jika kota tidak terhubung.
6. `show_cities()`: Menampilkan seluruh kota hub yang terdaftar sesuai urutan pendaftaran.
7. `show_routes()`: Menampilkan seluruh rute antar kota beserta jarak masing-masing.

3.5 Fungsi Utilitas

Program menyediakan sekumpulan fungsi bantu (helper) untuk kebutuhan input/output dan perhitungan:

1. `clear_screen()`: Mencetak garis pemisah untuk memberi kesan tampilan bersih pada CLI.
2. `pause()`: Menghentikan eksekusi sementara dan menunggu pengguna menekan ENTER, digunakan setelah menampilkan informasi.
3. `input_nonempty(prompt)`: Memastikan pengguna tidak memasukkan input kosong. Akan terus meminta ulang hingga input valid.
4. `input_float(prompt, minimum)`: Memvalidasi input numerik desimal dengan batas minimum opsional.
5. `input_int(prompt, minimum, maximum)`: Memvalidasi input bilangan bulat dengan batas minimum dan maksimum opsional.

6. `ask_yes_no(prompt)`: Meminta konfirmasi dari pengguna dengan pilihan Y (Ya) atau N (Tidak).
7. `format_money(amount)`: Memformat angka biaya menjadi format mata uang Rupiah (Rp x.xxx).
8. `calculate_badge_and_bonus(package_count)`: Menghitung lencana dan bonus insentif kurir berdasarkan jumlah paket yang ditangani. Terdapat empat kategori: Kurir Santai (0 paket, tanpa bonus), Kurir Reguler (1-3 paket, Rp25.000), Kurir Produktif (4-6 paket, Rp60.000), dan Kurir Elite (≥ 7 paket, Rp120.000).
9. `valid_resi_format(no_resi)`: Memvalidasi format nomor resi menggunakan regex. Resi harus tepat 4 digit numerik dan diawali angka '2'.

3.6 Fungsi `selection_sort_desc()`

Fungsi ini mengimplementasikan algoritma Selection Sort untuk mengurutkan daftar resi secara descending berdasarkan atribut `total_cost`. Cara kerja fungsi ini adalah membuat salinan list resi, kemudian untuk setiap iterasi *i*, mencari indeks elemen dengan `total_cost` terbesar dari posisi *i* hingga akhir, lalu menukarnya ke posisi *i*. Proses ini diulang hingga seluruh elemen terurut dari biaya terbesar ke terkecil.

3.7 Variabel Global dan Inisialisasi Data

Program menggunakan empat variabel global sebagai penyimpanan data utama yang diakses bersama oleh semua menu:

1. `graph`: Instansi `WeightedGraph` yang menyimpan seluruh data kota hub dan rute antar kota.
2. `bst`: Instansi `ReceiptBST` yang menyimpan seluruh data resi pengiriman.
3. `couriers`: List of Courier yang menyimpan data semua kurir terdaftar.
4. `manifest`: Dictionary dengan kunci `courier_id` dan nilai berupa list `no_resi` yang ditugaskan kepada kurir tersebut.
5. `assigned_resi`: Set yang menyimpan `no_resi` yang sudah dialokasikan ke kurir, mencegah satu resi ditugaskan ke lebih dari satu kurir.
6. Fungsi `seed_default_cities()` dipanggil di awal program untuk mendaftarkan kota hub default (SURABAYA, YOGYAKARTA) ke dalam `graph` secara otomatis.

BAB IV

ALUR PROGRAM DAN FUNGSI MENU

4.1 Fungsi main() dan Menu Utama

Fungsi main() adalah titik masuk program. Langkah pertama yang dilakukan adalah memanggil seed_default_cities() untuk menginisialisasi tiga kota default. Setelah itu, program masuk ke dalam loop utama yang menampilkan menu utama dengan empat pilihan:

1. Menu 1: Kelola Jaringan Hub dan Rute Logistik
2. Menu 2: Kelola Administrasi Resi Pengiriman
3. Menu 3: Kelola Kurir dan Manifest Pengantaran
4. Menu 0: Exit Program

Setiap pilihan memanggil fungsi sub-menu yang bersangkutan. Program terus berjalan hingga pengguna memilih opsi 0 untuk keluar.

```
=====
SILOG - SISTEM INFORMASI LOGISTIK
-----
1) Kelola Jaringan Hub dan Rute Logistik
2) Kelola Administrasi Resi Pengiriman
3) Kelola Kurir dan Manifest Pengantaran
0) Exit Program
Pilih menu utama: 1
```

4.2 Menu 1 – Kelola Jaringan Hub dan Rute Logistik

menu_1() mengelola semua operasi terkait graph. Sub-menu yang tersedia adalah:

1. 1.1 – input_city_menu(): Meminta nama kota baru dari pengguna dan memanggil graph.add_city(). Dilindungi validasi nama kosong dan duplikasi. Pengguna dapat menambahkan beberapa kota dalam satu sesi.

```
=====
MENU 1) KELOLA JARINGAN HUB DAN RUTE LOGISTIK
1.1) Input Hub Kota Baru
1.2) Input Rute Antar-Kota
1.3) Tampil Data Hub & Rute
0) Kembali ke MENU UTAMA
Pilih menu: 1.1

=====
Masukkan nama kota hub baru: SOLO
Hub kota berhasil ditambahkan.
Apakah ingin menambahkan kota lagi (Y/N)? Y

=====
Masukkan nama kota hub baru: SEMARANG
Hub kota berhasil ditambahkan.
Apakah ingin menambahkan kota lagi (Y/N)? N
```


2. 1.2 – input_route_menu(): Menampilkan daftar kota yang tersedia, lalu meminta input kota asal, kota tujuan, dan jarak. Memanggil graph.add_route() dan menampilkan pesan hasil. Program mewajibkan minimal 2 kota terdaftar sebelum rute dapat ditambahkan.

```
=====
MENU 1) KELOLA JARINGAN HUB DAN RUTE LOGISTIK
1.1) Input Hub Kota Baru
1.2) Input Rute Antar-Kota
1.3) Tampil Data Hub & Rute
0) Kembali ke MENU UTAMA
Pilih menu: 1.2

=====

Daftar Kota Hub:
+-----+-----+
| No | Nama Kota |
+-----+-----+
| 1 | SURABAYA |
| 2 | YOGYAKARTA |
| 3 | SOLO |
| 4 | SEMARANG |
+-----+-----+
Masukkan kota asal: SURABAYA
Masukkan kota tujuan: YOGYAKARTA
Masukkan jarak rute (KM): 300
Rute berhasil ditambahkan.
Apakah ingin menambahkan rute lagi (Y/N)? Y

=====
```

3. 1.3 – Tampil Data Hub & Rute: Memanggil graph.show_cities() dan graph.show_routes() untuk menampilkan seluruh data jaringan logistik yang sudah terdaftar.

```
=====
MENU 1) KELOLA JARINGAN HUB DAN RUTE LOGISTIK
1.1) Input Hub Kota Baru
1.2) Input Rute Antar-Kota
1.3) Tampil Data Hub & Rute
0) Kembali ke MENU UTAMA
Pilih menu: 1.3

Daftar Kota Hub:
+-----+-----+
| No | Nama Kota |
+-----+-----+
| 1 | SURABAYA |
| 2 | YOGYAKARTA |
| 3 | SOLO |
| 4 | SEMARANG |
+-----+-----+

Daftar Rute Antar-Kota:
+-----+-----+-----+-----+
| No | Kota Asal | Kota Tujuan | Jarak |
+-----+-----+-----+-----+
| 1 | SURABAYA | YOGYAKARTA | 300 KM |
| 2 | YOGYAKARTA | SURABAYA | 300 KM |
|   |   | SOLO | 80 KM |
|   |   | SEMARANG | 150 KM |
| 3 | SOLO | YOGYAKARTA | 80 KM |
| 4 | SEMARANG | YOGYAKARTA | 150 KM |
+-----+-----+-----+-----+

Tekan ENTER untuk kembali...

=====
```

4.3 Menu 2 – Kelola Administrasi Resi Pengiriman

menu_2() mengelola semua operasi terkait resi menggunakan BST. Sub-menu yang tersedia adalah:

1. 2.1 – input_receipt_menu(): Memandu pengguna memasukkan data resi baru. Proses validasi bertahap dilakukan: format dan keunikan no_resi dicek terlebih dahulu, lalu kota asal dan tujuan diverifikasi keberadaannya di graph, kemudian Dijkstra dipanggil untuk mendapatkan jarak terpendek. Jika kota tidak terhubung, resi tidak dapat dibuat. Setelah jarak diperoleh, biaya dihitung otomatis dan data disimpan ke BST.

```
Pilih menu utama: 2

=====
MENU 2) KELOLA ADMINISTRASI RESI PENGIRIMAN
2.1) Input Resi Pengiriman Baru
2.2) Lihat Seluruh Data Resi Terdaftar
2.3) Urutkan Transaksi Resi Berdasarkan Biaya Terbesar
0) Kembali ke MENU UTAMA
Pilih menu: 2.1

=====
INPUT RESI PENGIRIMAN BARU
Masukkan No. Resi (4 digit, diawali '2'): 2001
Nama Pengirim: SYAYIDATI
Kota Asal: YOGYAKARTA
Kota Tujuan: SURABAYA
Berat Paket (Kg): 1

Input Resi Berhasil!
+-----+-----+-----+-----+-----+-----+-----+
| No Resi | Pengirim      | Asal      | Tujuan    | Berat (Kg) | Jarak (KM) | Total Biaya |
+-----+-----+-----+-----+-----+-----+-----+
| 2001    | SYAYIDATI     | YOGYAKARTA | SURABAYA  | 1          | 300        | Rp 605.000  |
+-----+-----+-----+-----+-----+-----+-----+

Apakah ingin melakukan input resi lagi (Y/N)? Y
```

2. 2.2 – show_all_receipts_menu(): Menampilkan seluruh resi yang tersimpan dalam BST melalui traversal inorder, sehingga data tampil terurut berdasarkan no_resi secara alfabetis.

```
Pilih menu: 2.2

=====
DATA RESI TERDAFTAR
+-----+-----+-----+-----+-----+-----+-----+
| No Resi | Pengirim      | Asal      | Tujuan    | Berat (Kg) | Jarak (KM) | Total Biaya |
+-----+-----+-----+-----+-----+-----+-----+
| 2001    | SYAYIDATI     | YOGYAKARTA | SURABAYA  | 1          | 300        | Rp 605.000  |
| 2002    | HAPSARI       | YOGYAKARTA | SOLO      | 2          | 80         | Rp 170.000  |
| 2003    | FAIRUZ        | SEMARANG   | YOGYAKARTA | 4          | 150        | Rp 320.000  |
| 2004    | RAHEL         | SOLO       | YOGYAKARTA | 1          | 80         | Rp 165.000  |
+-----+-----+-----+-----+-----+-----+-----+

Tekan ENTER untuk kembali...
```

3. 2.3 – sort_receipts_menu(): Mengambil semua data resi dari BST, lalu mengurutkannya menggunakan selection_sort_desc() dan menampilkan hasilnya dari biaya terbesar ke terkecil.

Pilih menu: 2.3

```
=====
URUTAN TRANSAKSI RESI BERDASARKAN BIAYA TERBESAR
+-----+-----+-----+-----+-----+-----+-----+
| Rank | No Resi | Total Biaya | Pengirim | Asal | Tujuan | Berat (Kg) |
+-----+-----+-----+-----+-----+-----+-----+
| 1 | 2001 | Rp 605.000 | SYAYIDATI | YOGYAKARTA | SURABAYA | 1 |
| 2 | 2003 | Rp 320.000 | FAIRUZ | SEMARANG | YOGYAKARTA | 4 |
| 3 | 2002 | Rp 170.000 | HAPSARI | YOGYAKARTA | SOLO | 2 |
| 4 | 2004 | Rp 165.000 | RAHEL | SOLO | YOGYAKARTA | 1 |
+-----+-----+-----+-----+-----+-----+-----+
```

Tekan ENTER untuk kembali...

4.4 Menu 3 – Kelola Kurir dan Manifest Pengantaran

menu_3() mengelola semua operasi terkait kurir dan penugasan resi. Sub-menu yang tersedia adalah:

1. 3.1 – input_courier_menu(): Meminta data kurir baru (ID, nama, jenis kendaraan). ID yang duplikat ditolak. Jenis kendaraan dibatasi pada Motor, Mobil, atau Truck. Setelah kurir berhasil ditambahkan ke list couriers, entri kosong untuk manifest juga disiapkan.

Pilih menu utama: 3

```
=====
MENU 3) KELOLA KURIR DAN MANIFEST PENGANTARAN
3.1) Input Data Kurir
3.2) Plotting Penugasan Manifest
3.3) Tampil Manifest & Aturan Bonus Insentif
0) Kembali ke MENU UTAMA
Pilih menu: 3.1
```

```
=====
INPUT DATA KURIR
ID Kurir: 001
Nama Kurir: BUDI
Jenis Kendaraan (Motor/Mobil/Truck): MOTOR
Data Kurir Berhasil Diinputkan
Apakah ingin menambahkan data Kurir lagi (Y/N)? Y
```

2. 3.2 – plotting_manifest_menu(): Menampilkan daftar kurir dan daftar resi beserta statusnya (tersedia atau sudah dialokasikan). Pengguna memilih ID kurir, lalu memasukkan no_resi yang akan ditugaskan. Validasi dilakukan untuk memastikan resi ada di BST dan belum ditugaskan ke kurir lain. no_resi yang berhasil ditugaskan ditambahkan ke manifest dan dicatat di assigned_resi.

Pilih menu: 3.2

=====

PLOTING PENUGASAN MANIFEST

Daftar Kurir:

ID Kurir	Nama	Kendaraan
001	BUDI	Motor
002	DUDUNG	Mobil
003	UCUP	Mobil

Masukkan ID Kurir yang akan ditugaskan: 002

Kurir terpilih: DUDUNG (Mobil)

Silakan pilih resi yang akan dimasukkan ke manifest.

Daftar Resi:

No Resi	Asal	Tujuan	Total Biaya	Status
2001	YOGYAKARTA	SURABAYA	Rp 605.000	Tersedia
2002	YOGYAKARTA	SOLO	Rp 170.000	Tersedia
2003	SEMARANG	YOGYAKARTA	Rp 320.000	Tersedia
2004	SOLO	YOGYAKARTA	Rp 165.000	Tersedia

Masukkan No. Resi yang akan ditugaskan: 2001

Resi berhasil ditambahkan ke manifest kurir.

Apakah ingin memasukkan No. Resi lagi (Y/N)? Y

3. 3.3 – tampil_manifest_menu(): Menampilkan laporan lengkap setiap kurir, mencakup identitas kurir, lencana dan bonus insentif berdasarkan jumlah resi, serta detail setiap resi dalam manifest kurir tersebut.

Pilih menu: 3.3

=====

TAMPIL MANIFEST & ATURAN BONUS INSENTIF

No	ID Kurir	Nama Kurir	Kendaraan	Jml Resi	Lencana	Bonus Insentif
1	001	BUDI	Motor	1	Kurir Reguler	Rp 25.000
Rincian Resi:						
No	No Resi	Pengirim	Asal	Tujuan	Berat	Total Biaya
1	2005	LORENZA	YOGYAKARTA	SOLO	1 Kg	Rp 165.000
No	ID Kurir	Nama Kurir	Kendaraan	Jml Resi	Lencana	Bonus Insentif
2	002	DUDUNG	Mobil	2	Kurir Reguler	Rp 25.000
Rincian Resi:						
No	No Resi	Pengirim	Asal	Tujuan	Berat	Total Biaya
1	2001	SYAYIDATI	YOGYAKARTA	SURABAYA	1 Kg	Rp 605.000
2	2003	FAIRUZ	SEMARANG	YOGYAKARTA	4 Kg	Rp 320.000

BAB V

ALUR DATA DAN KETERKAITAN ANTAR KOMPONEN

5.1 Alur Data Utama

Ketiga modul utama program saling berkaitan membentuk alur logis yang kohesif. Berikut adalah alur data secara keseluruhan:

1. Pengguna mendaftarkan kota-kota hub dan rute antar kota melalui Menu 1. Data ini tersimpan dalam graph (WeightedGraph).
2. Saat pengguna membuat resi baru di Menu 2, program secara otomatis menjalankan algoritma Dijkstra pada graph untuk mendapatkan jarak terpendek antara kota asal dan tujuan. Jika tidak ada jalur, resi tidak dapat dibuat.
3. Jarak hasil Dijkstra digunakan untuk menghitung biaya pengiriman (total_cost) yang kemudian disimpan bersama objek Receipt ke dalam BST.
4. Di Menu 3, kurir didaftarkan dan dapat diberikan penugasan resi dari BST. Program mencegah duplikasi penugasan menggunakan set assigned_resi.
5. Laporan manifest menggabungkan data dari couriers, manifest, dan BST untuk menampilkan informasi lengkap termasuk insentif kurir.

5.2 Keterkaitan Komponen

Komponen	Struktur Data	Peran dalam Sistem
WeightedGraph	Adjacency List (Dict)	Menyimpan jaringan kota hub dan rute, sumber data jarak untuk BST
ReceiptBST	Binary Search Tree	Menyimpan dan mencari resi, sumber data untuk manifest
couriers	List of Dataclass	Menyimpan identitas kurir
manifest	Dictionary of List	Memetakan kurir ke daftar resi yang ditugaskan
assigned_resi	Set	Mencegah duplikasi penugasan resi ke kurir
selection_sort_desc()	In-place Array Sort	Mengurutkan resi berdasarkan biaya
Dijkstra (shortest_distance)	Priority Queue (heapq)	Menghitung jarak terpendek antar kota

BAB VI

PENUTUP

6.1 Kesimpulan

Proyek SILOG berhasil mengimplementasikan sistem informasi logistik berbasis struktur data dengan memanfaatkan Weighted Undirected Graph untuk pengelolaan jaringan rute, Binary Search Tree untuk administrasi resi, serta Array dan Dictionary untuk manajemen kurir dan manifest.

Algoritma Dijkstra diintegrasikan langsung ke dalam proses pembuatan resi sehingga sistem mampu menolak pengiriman ke kota yang tidak terhubung secara otomatis. Penerapan Selection Sort secara descending memungkinkan tampilan transaksi berdasarkan biaya tertinggi yang berguna untuk analisis prioritas pengiriman.

Secara keseluruhan, ketiga struktur data yang digunakan saling melengkapi dan terintegrasi dengan baik, membentuk sistem yang fungsional dan sesuai dengan persyaratan proyek final struktur data.

6.2 Saran

1. Program dapat dikembangkan dengan menambahkan fitur penghapusan dan pembaruan data resi atau kurir.
2. Antarmuka dapat ditingkatkan menjadi GUI (Graphical User Interface) untuk kemudahan penggunaan.
3. Fitur penyimpanan data ke file (misalnya JSON atau CSV) dapat ditambahkan agar data tidak hilang saat program ditutup.